



PONTIFICIA UNIVERSIDAD CATÓLICA DE CHILE
ESCUELA DE INGENIERÍA
DEPARTAMENTO DE CIENCIA DE LA COMPUTACIÓN

IIC2223 — Teoría de Autómatas y Lenguajes Formales — 2° 2018

PAUTA EXAMEN

Pregunta 1

Pregunta 1.1

Para esta pregunta, era necesario calcular los conjuntos FIRST y FOLLOW de cada variable en la gramática primero, donde obtenemos:

- Como todas las producciones salientes de T comienzan en terminales o son en vacío, entonces $\text{FIRST}(T) = \{a, c, \epsilon\}$.
- En el caso de S , también tiene una producción en vacío, pero también tiene una que comienza con T . Por lo último, esto agrega los terminales de $\text{FIRST}(T)$ en $\text{FIRST}(S)$. El hecho de que $\epsilon \in \text{FIRST}(T)$ no agrega más información ya que el símbolo que continua en la producción es el mismo S . Luego, $\text{FIRST}(S) = \{a, c, \epsilon\}$.
- Como S es la variable inicial, $\$ \in \text{FOLLOW}(S)$. Por otro lado, hay dos apariciones de S en producciones de T , donde en cada una S es seguida de terminales (b y d). Finalmente, S también aparece al final de una producción desde ella misma, lo cual no agrega información ($\text{FOLLOW}(S) \subseteq \text{FOLLOW}(S)$). Luego, $\text{FOLLOW}(S) = \{\$, b, d\}$.
- En el caso de T , su única aparición es en $S \rightarrow TS$. Luego, como la sigue una S , entonces $\text{FIRST}(S) \cap \Sigma \subseteq \text{FOLLOW}(T)$. Además, como $\epsilon \in \text{FIRST}(S)$, hay que considerar el siguiente símbolo, pero ahí termina la producción, entonces hay que incluir lo que puede ir después de S , al ser la variable a la izquierda de la producción: $\text{FOLLOW}(S) \subseteq \text{FOLLOW}(T)$. Lo cual resulta en: $\text{FOLLOW}(S) = \{\$, a, b, c, d\}$

Para verificar si la gramática es LL(1), hay que verificar que para cada par de producciones con igual variable a la izquierda cumpliera con las condiciones de LL(1). Uno podía notar que no cumplía por alguna de las siguientes violaciones:

- Para $S \rightarrow TS \mid \epsilon$, como $\epsilon \in \text{FIRST}(T) \cap \text{FIRST}(S)$, entonces $\epsilon \in \text{FIRST}(TS)$. Como $\text{FIRST}(TS) \cap \text{FIRST}(\epsilon) = \{\epsilon\} \neq \emptyset$, se viola la primera condición de LL(1).
- Para $T \rightarrow aSb \mid \epsilon$, como $\epsilon \in \text{FIRST}(\epsilon)$ y $\{a\} = \text{FIRST}(aSb)$, notamos que $\text{FIRST}(aSb) \cap \text{FOLLOW}(T) = \{a\} \neq \emptyset$. Lo cual viola la segunda y tercera propiedad de LL(1).
- Para $T \rightarrow cSd \mid \epsilon$, de forma análoga a la violación anterior con el terminal c .

Dado lo anterior, el puntaje asignado es el siguiente:

- **(0.5 puntos)**: Por el correcto computo de $\text{FIRST}(T)$.
(0.25 puntos): Por fallar en incluir algunos componentes.
(0 puntos): Por computarlos incorrectamente.
- **(0.5 puntos)**: Por el correcto computo de $\text{FIRST}(S)$.
(0.25 puntos): Por fallar en incluir algunos componentes.
(0 puntos): Por computarlos incorrectamente.
- **(0.5 puntos)**: Por el correcto computo de $\text{FOLLOW}(T)$.
(0.25 puntos): Por fallar en incluir algunos componentes.
(0 puntos): Por computarlos incorrectamente.
- **(0.5 puntos)**: Por el correcto computo de $\text{FOLLOW}(S)$.
(0.25 puntos): Por fallar en incluir algunos componentes.
(0 puntos): Por computarlos incorrectamente.
- **(1 punto)**: Por encontrar una violación de LL(1) y decidir que no es de ese tipo.
(0.5 punto): por aplicar correctamente las condiciones con computos incorrectos que llevarían a decidir incorrectamente si es LL(1).
(0 puntos): Por incorrecta decisión y mal aplicación de las condiciones.

Pregunta 1.2

Esta pregunta podía resolverse mediante el algoritmo visto el clases de eliminación de recursión por la izquierda, o por algún método alternativo que generara una gramática equivalente.

Usando el algortimo visto en clases, el puntaje asignado es el siguiente:

- **(1 punto)**: Por aplicar correctamente eliminación de recursividad inmediata en algún punto de su procedimiento.
- **(0.5 puntos)**: Por realizar correctamente composición de variables en producciones para simplificar recursión no-inmediata, en particular, componer S y R de alguna forma.
- **(0.5 puntos)**: Por realizar el paso de algoritmo para T .
- **(0.5 puntos)**: Por realizar el paso de algoritmo para R .
- **(0.5 puntos)**: Por realizar el paso de algoritmo para S .

En los casos alternativos sin usar el método de clases:

- **(1.5 puntos)**: Por construcción de gramática equivalente.
- **(1.5 puntos)**: Por explicación de pasos que expliquen porque es equivalente a la original (como eliminar recursión inmediata, simplificar producciones y composición de variables)

Pregunta 2

Pregunta 2.1

Como L es regular, existe un DFA $\mathcal{A} = (Q, \Sigma, \delta, q_0, F)$ tal que $\mathcal{L}(\mathcal{A}) = L$. Luego, se define la CFG $\mathcal{G} = (V, \Sigma, P, S)$ para $\mathcal{A}$ con:

- $V = Q$,

- $P = \{p \rightarrow aq \mid (p, a, q) \in \delta\} \cup \{p \rightarrow \epsilon \mid p \in F\}$, y
- $S = q_0$

Finalmente, basta demostrar que $\mathcal{G}$ es LL(1), para lo que deben demostrar que, para todo par de producciones $p \rightarrow \alpha \mid \beta$:

1. $\text{FIRST}(\alpha) \cap \text{FIRST}(\beta) = \emptyset$,
2. si $\epsilon \in \text{FIRST}(\alpha)$, entonces $\text{FIRST}(\alpha) \cap \text{FOLLOW}(A) = \emptyset$, y
3. si $\epsilon \in \text{FIRST}(\beta)$, entonces $\text{FIRST}(\beta) \cap \text{FOLLOW}(A) = \emptyset$.

Dado lo anterior, el puntaje asignado es el siguiente:

- **(0.5 puntos)**: Por decir que existe DFA $\mathcal{A}$ para el lenguaje L .
- **(1 punto)**: Por definir la gramática $\mathcal{G}$ para $\mathcal{A}$.
- **(1 punto)**: Por demostrar que $\mathcal{G}$ cumple con la primera condición de LL(1).
- **(0.5 puntos)**: Por demostrar que $\mathcal{G}$ cumple con la segunda y tercera condición de LL(1).

Pregunta 2.2

Dada la gramática $\mathcal{G}$, el proceso de remoción de producciones unitarias tiene dos pasos:

1. definir el conjunto P^* (mínimo) de producciones que contiene a P tal que, si $X \rightarrow Y \in P^*$ y $Y \rightarrow \alpha \in P^*$, entonces $X \rightarrow \alpha \in P^*$,
2. definir el conjunto $\hat{P}$ que remueve todas las producciones unitarias de P^* .

Como $\mathcal{G}$ no tiene producciones en vacío, tampoco las tendrá $\hat{\mathcal{G}}$. Por lo tanto, $\hat{\mathcal{G}}$ cumple trivialmente las condiciones 2 y 3 de LL(1), y basta demostrar que para cualquier par de producciones $X \rightarrow \alpha \mid \beta \in \hat{P}$, $\text{FIRST}(\alpha) \cap \text{FIRST}(\beta) = \emptyset$.

Por construcción, $X \rightarrow \alpha \in \hat{P}$ implica que en P hay una serie de producciones $X \rightarrow Y_1 \rightarrow Y_2 \rightarrow \dots \rightarrow Y_n \rightarrow \alpha$. A su vez, esto implica que $\text{FIRST}(\alpha) \subseteq \text{FIRST}(Y_n) \subseteq \dots \subseteq \text{FIRST}(Y_2) \subseteq \text{FIRST}(Y_1)$. De la misma forma, $X \rightarrow \beta \in \hat{P}$ implica que en P hay una serie de producciones $X \rightarrow Z_1 \rightarrow Z_2 \rightarrow \dots \rightarrow Z_m \rightarrow \beta$ y, por lo tanto, $\text{FIRST}(\beta) \subseteq \text{FIRST}(Z_m) \subseteq \dots \subseteq \text{FIRST}(Z_2) \subseteq \text{FIRST}(Z_1)$. Luego, como $\mathcal{G}$ es LL(1), cumple que $\text{FIRST}(Y_1) \cap \text{FIRST}(Z_1) = \emptyset$ y, por las contenciones anteriores, $\text{FIRST}(\alpha) \cap \text{FIRST}(\beta) = \emptyset$. Por lo tanto, $\hat{\mathcal{G}}$ es LL(1).

Dado lo anterior, el puntaje asignado es el siguiente:

- **(0.5 puntos)**: Por definir correctamente el conjunto $\hat{P}$ (agregar primero todas las producciones de P^* y luego borrar las unitarias).
- **(0.5 puntos)**: Por decir que las condiciones 2 y 3 de LL(1) se cumplen al no tener producciones en vacío, o por decir que la función FIRST no varía de $\mathcal{G}$ a $\hat{\mathcal{G}}$.
- **(1 punto)**: Por decir que existen las series de producciones $X \rightarrow Y_1 \rightarrow \dots \rightarrow Y_n \rightarrow \alpha$ y $X \rightarrow Z_1 \rightarrow \dots \rightarrow Z_m \rightarrow \beta$ en P .
- **(1 punto)**: Por demostrar que se cumple la condición 1.

Si la definición de $\hat{P}$ está incorrecta pero es razonable (por ejemplo, diciendo que si $X \rightarrow Y \in P$ y $Y \rightarrow \alpha \in P$, entonces se agrega $X \rightarrow \alpha \in \hat{P}$ y se borra $X \rightarrow Y$), se asignó un puntaje máximo de **1.5 puntos** si la demostración era correcta y si se siguió un proceso similar al de la pauta.

Pregunta 3

Pregunta 3.1

Este lenguaje no es regular, por lo tanto una posible solución consistía en utilizar el contrapositivo del lema de bombeo para lenguajes regulares. Para esto, para cualquier $N > 0$, sea la palabra $a \cdot b^N \cdot c^N$, la cual descomponemos en xyz donde:

- $x = a$
- $y = b^N$
- $z = c^N$

Claramente $|y| \geq N$, por lo que la descomposición es correcta. Luego, cualquier descomposición $uvw = y$ que haga el adversario, se verá de la forma:

- $u = b^j$
- $v = b^k$
- $w = b^l$

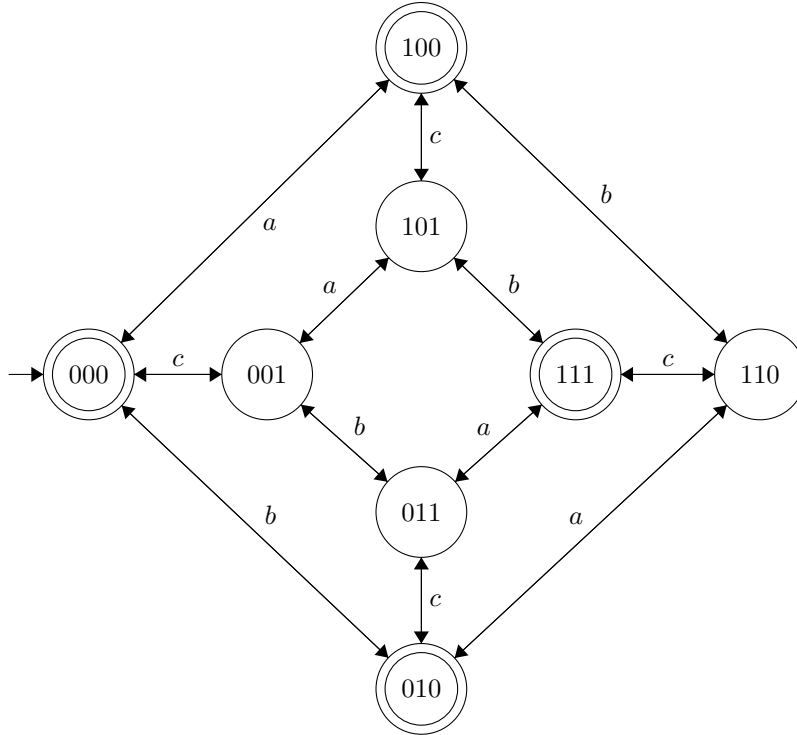
Donde $j + k + l = N$ y $k > 0$. Por lo tanto, sea $i = 0$. Como $k > 0$, tenemos que $j + l < N$ y por lo tanto la palabra generada por el bombeo, $a \cdot b^j \cdot b^l \cdot c^N$ no pertenece al lenguaje.

Dado lo anterior, el puntaje asignado es el siguiente:

- **(1.5 puntos):** Por escoger y dividir la palabra correctamente.
- **(1.5 puntos):** Por aplicar el bombeo y argumentar que la palabra resultante no está en el lenguaje

Pregunta 3.2

Este lenguaje si es regular, por lo tanto una posible solución es entregar un autómata finito que acepte el lenguaje. La intuición está en que no es necesario recordar la cantidad exacta de cada letra, sino que simplemente basta con su paridad. Esto claramente es una cantidad finita de información y por lo tanto puede ser representada por un DFA. Se propone el siguiente autómata, donde cada nombre de estado representa la paridad de letras a , b y c respectivamente, donde un 0 significa que es par y un 1 lo contrario.



Dado lo anterior, el puntaje asignado es el siguiente:

- **(2 puntos):** Por entregar la construcción del autómata
- **(1 punto):** Por explicar la construcción del autómata

Pregunta 4

Una posible solución era cambiar el conjunto P por un diccionario que asocie cada estado con el número de ejecuciones que pueden terminar en él tras leer cada prefijo de la palabra w . Tras leer toda la palabra, la respuesta es la suma de los valores asociados a los estados finales. En pseudo-python sería:

```

function COUNT-NFAONTHEFLY( $\mathcal{A} = (Q, \Sigma, \Delta, I, F), w = a_1 \cdots a_n$ )
   $P = \{q : 0 \text{ for } q \text{ in } Q\}$ 
  for  $q \in I$  do
     $P[q] = 1$ 
  for  $i = 1$  to  $|w|$  do
     $P_t = \{q : 0 \text{ for } q \text{ in } Q\}$ 
    for  $(p, a_i, q) \in \Delta$  do
       $P_t[q] = P_t[q] + P[p]$ 
     $P = P_t$ 
  return  $\sum_{q \in F} P[q]$ 

```

Dado lo anterior, el puntaje asignado es el siguiente:

- **(4 puntos):** Por entregar un algoritmo correcto que funcione en tiempo polinomial en la palabra y el automata.

- **(2 puntos)**: Por explicar la idea detrás del algoritmo (siempre y cuando cumpla con las condiciones anteriores).

Pregunta 5

Dado que L es regular entonces existe un DFA $\mathcal{A} = (Q, \Sigma, \delta, q_0, F)$ tal que $\mathcal{L}(\mathcal{A}) = L$. De la misma manera, como R es libre de contexto existe un PDA $\mathcal{P} = (P, \Sigma, \Gamma, \Delta, q'_0, \perp, F')$ tal que $\mathcal{L}_F(\mathcal{P}) = R$. La idea será definir un nuevo autómata apilador que primero simule la ejecución de $\mathcal{A}$ y que en un cierto momento comience a simular una ejecución de $\mathcal{P}$ simultánea a la de $\mathcal{A}$. Una solución es el siguiente autómata apilador:

$$\mathcal{P}' = (Q \cup (Q \times P), \Sigma, \Gamma, \Delta', q_0, \perp', F \times F')$$

Con Δ' tal que:

- para toda transición $\delta(p, a) = q$ se tiene que $(p \perp' \xrightarrow{a} q \perp') \in \Delta'$
- para todo $p \in Q$, $(p \perp' \xrightarrow{\epsilon} (p, q'_0) \perp) \in \Delta'$
- para toda transición $\delta(p, a) = q$ y $(p'A \xrightarrow{a} q'\gamma) \in \Delta$ con $a \in \Sigma$ se tiene $((p, p')A \xrightarrow{a} (q, q')\gamma) \in \Delta'$
- para toda transición $(p'A \xrightarrow{\epsilon} q'\gamma) \in \Delta$ se tendrá que, para todo $p \in Q$, $((p, p')A \xrightarrow{\epsilon} (p, q')\gamma) \in \Delta'$

Luego se debe demostrar que $\mathcal{L}_F(\mathcal{P}') = L \sqcup R$ mediante el análisis de la ejecución del autómata.

Dado lo anterior, el puntaje asignado es el siguiente:

- **(0.5 puntos)**: Por mencionar que el conjunto de estados finales de $\mathcal{P}'$ es el producto cartesiano entre los estados finales de $\mathcal{A}$ y $\mathcal{P}$.
- **(1 punto)**: Por la noción de que primero se debe ejecutar $\mathcal{A}$ normalmente sobre la palabra.
- **(1 punto)**: Por explicitar que debe existir un salto no-determinista que salte a la ejecución simultánea de $\mathcal{A}$ con $\mathcal{P}$.
- **(2 puntos)**: Por definir como se realiza la ejecución simultánea de $\mathcal{A}$ y $\mathcal{P}$ cuando se lee una letra $a \in \Sigma$ cualquiera.
- **(0.5 puntos)**: Por definir como se realiza la ejecución simultánea de $\mathcal{A}$ y $\mathcal{P}$ cuando en $\mathcal{P}$ hay una ϵ -transición.
- **(1 punto)**: Por demostrar que $\mathcal{L}(\mathcal{P}') = L \sqcup R$.